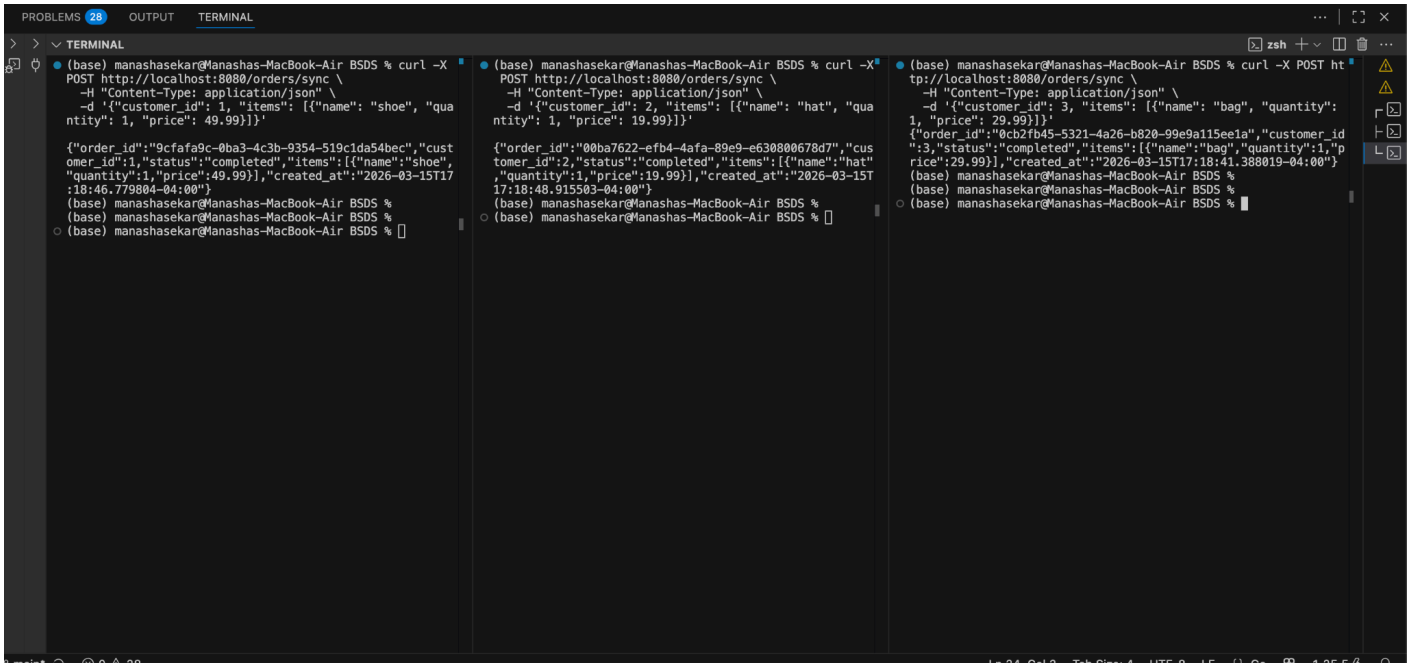
Name: Manasha Sekar

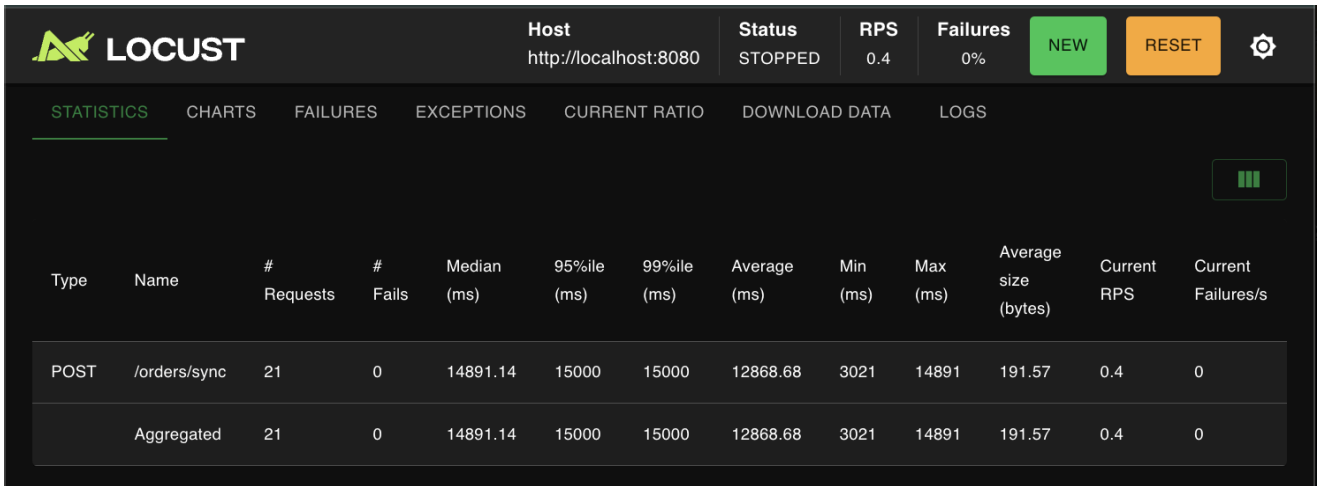
Phase 1

Local curl test:

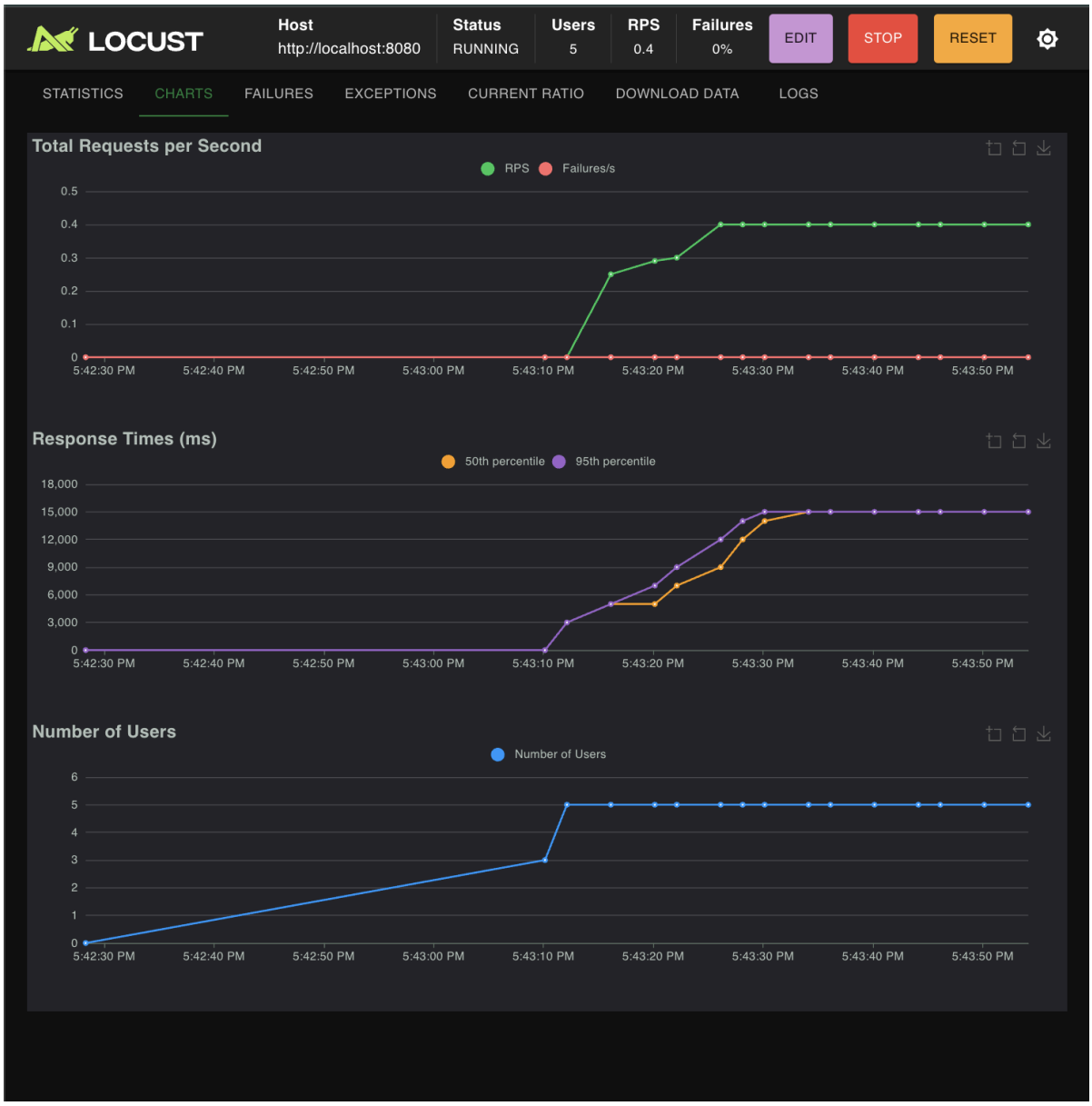


I sent the tests simultaneously but the responses took time to reach, even when requests are sent parallely is processed in sequence and then the results are returned.

Normal Test: 5 users for 30 secs.

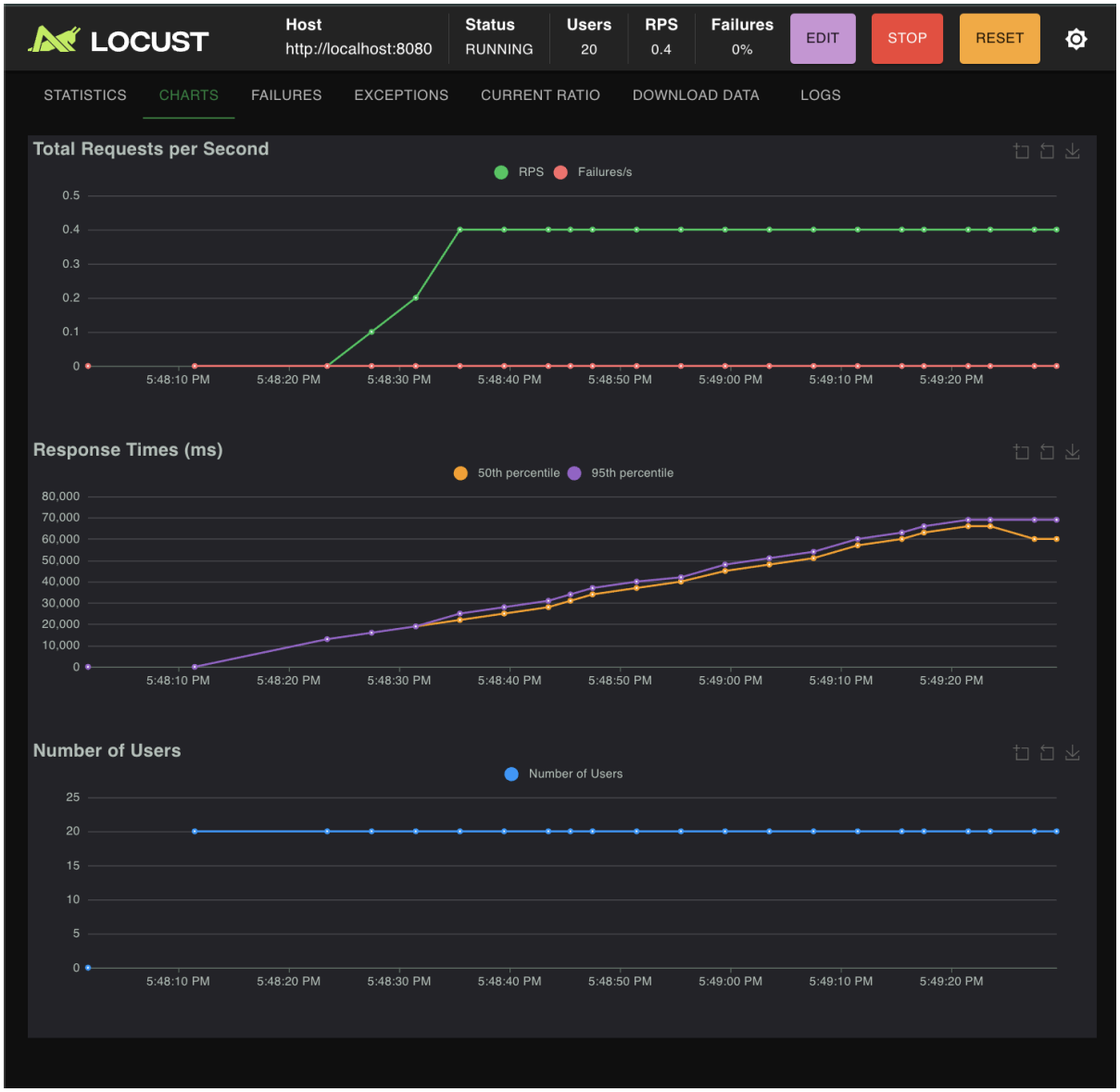


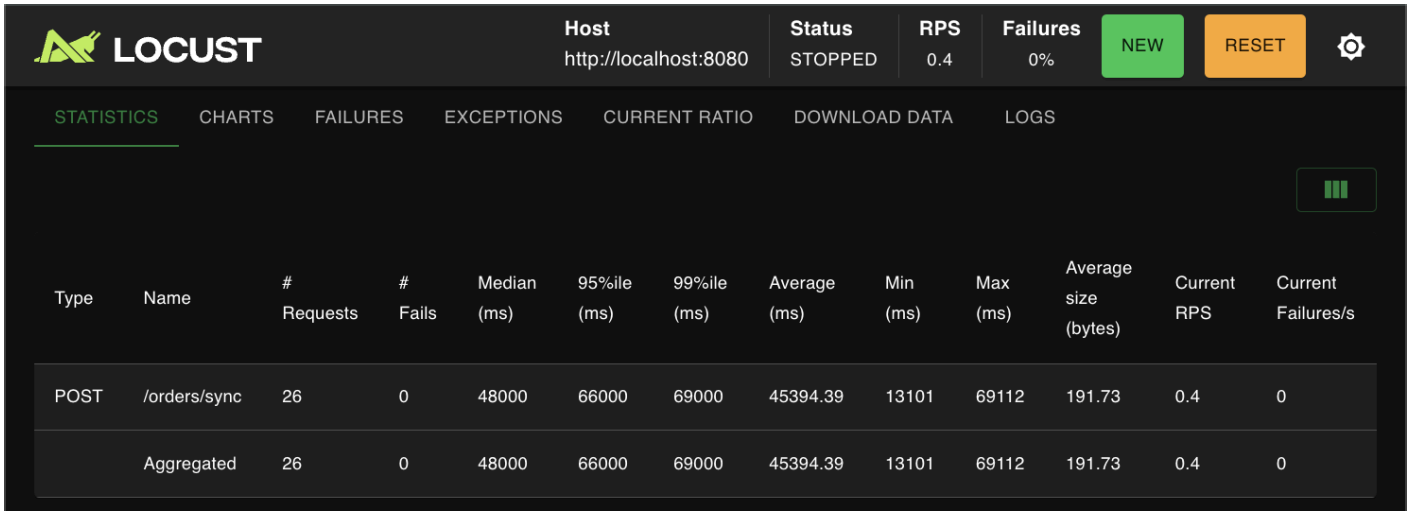
Name: Manasha Sekar



Name: Manasha Sekar

Flash sale for 20 users 60 secs.





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	26	0	48000	66000	69000	45394.39	13101	69112	191.73	0.4	0
	Aggregated	26	0	48000	66000	69000	45394.39	13101	69112	191.73	0.4	0

Name: Manasha Sekar

Comparison:

Test	Users	RPS	Failures	95th percentile
Normal	5	0.4	0%	~15,000ms
Flash Sale	20	0.4	0%	~70,000ms

The system does not crashes it just makes the customers wait forever!

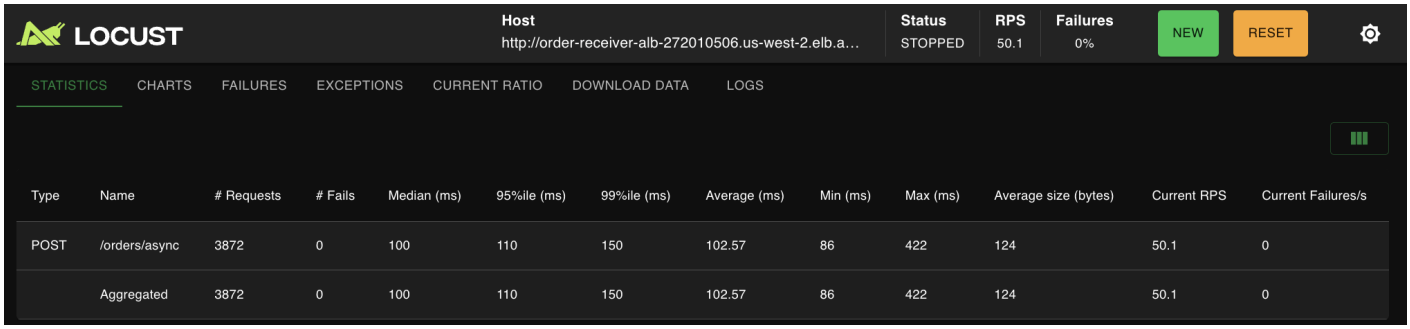
Phase 2: Analyze the Bottleneck

Payment processor speed: 1 order per 3 seconds
With 20 concurrent customers: Maximum throughput = 0.33 orders/second
Flash sale demand: 20 orders/second
Orders lost per second: 19.67 orders

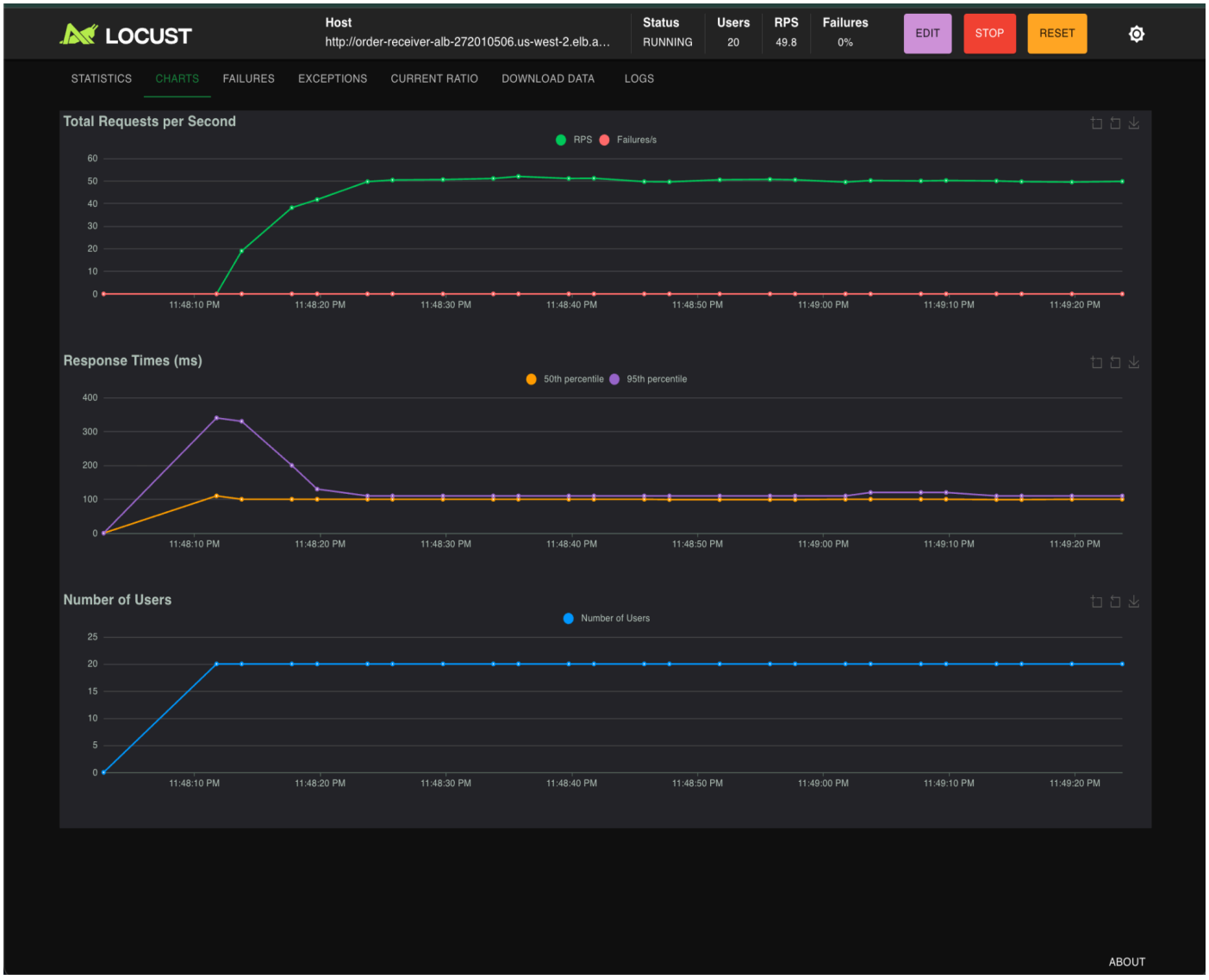
Name: Manasha Sekar

Phase 3: The Async Solution

Worker:1

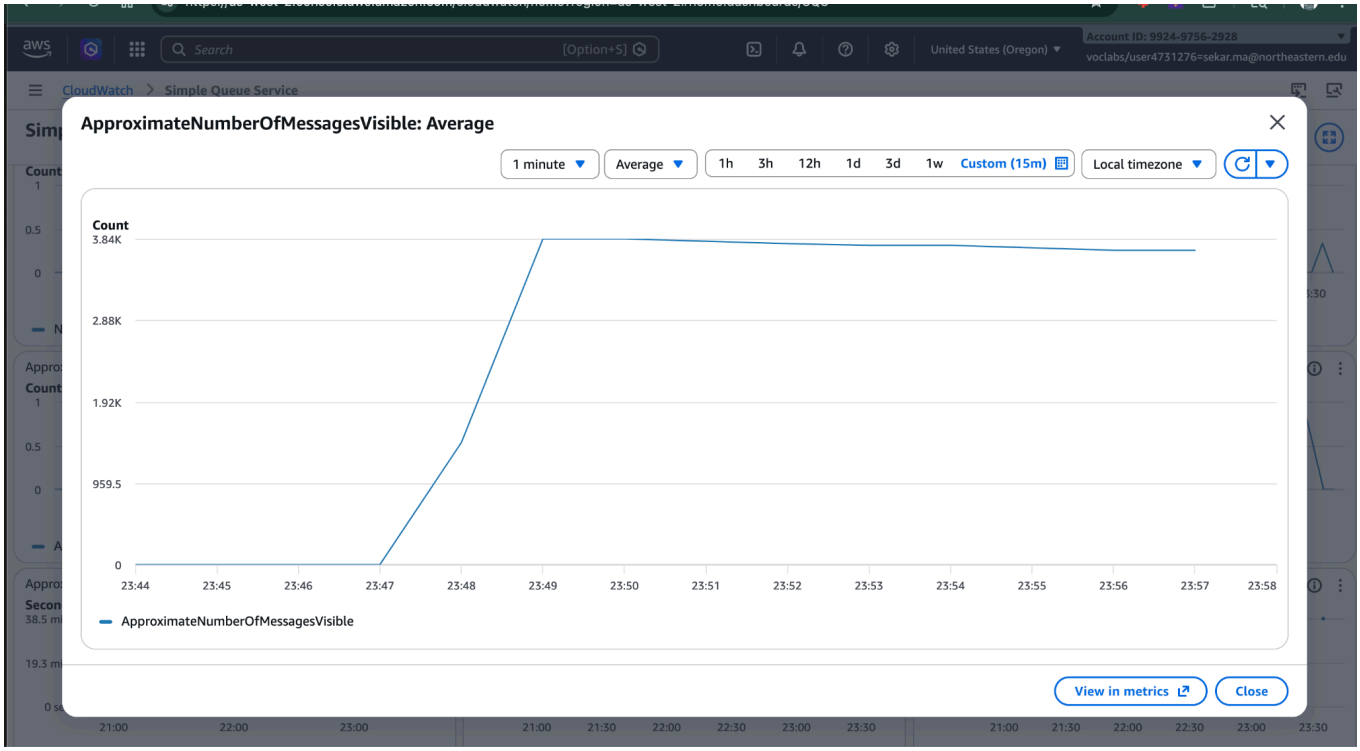


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3872	0	100	110	150	102.57	86	422	124	50.1	0
	Aggregated	3872	0	100	110	150	102.57	86	422	124	50.1	0



Cloud Watch:

Name: Manasha Sekar



Test Configuration: 20 users, 60 seconds, spawn rate 10/s

Metric	Sync (Flash Sale)	Async (Flash Sale)
RPS	0.4	49.9
Failures	0%	0%
Avg Response Time	~45,000ms	~102ms
Orders Accepted	26	3,872

The async endpoint accepted **3,872 orders** compared to just **26** with the synchronous approach — roughly **149x more orders** in the same time window. Response times dropped from an average of 45 seconds down to 102ms, meaning customers received an immediate acknowledgment instead of waiting for payment verification to complete.

This is the core trade-off of async processing: the API returns a `202 Accepted` instantly and delegates the actual work to background workers. Customers are never blocked, and the system can absorb any burst of traffic regardless of how slow the downstream payment processor is.

Phase 4

Name: Manasha Sekar

Analysis:

- Order acceptance rate: ~50/second
- Single worker processing rate: 1 order per 3 seconds = 0.33/second
- Queue growth rate: 49.67 messages/second
- Time to clear backlog: ~192 minutes

Observations

While the async endpoint successfully accepted all 3,872 orders with 0% failures, the single background worker could only process one order every 3 seconds. During the 60-second flash sale, the queue grew at roughly 49.67 messages per second, peaking at approximately 3,840 visible messages in CloudWatch.

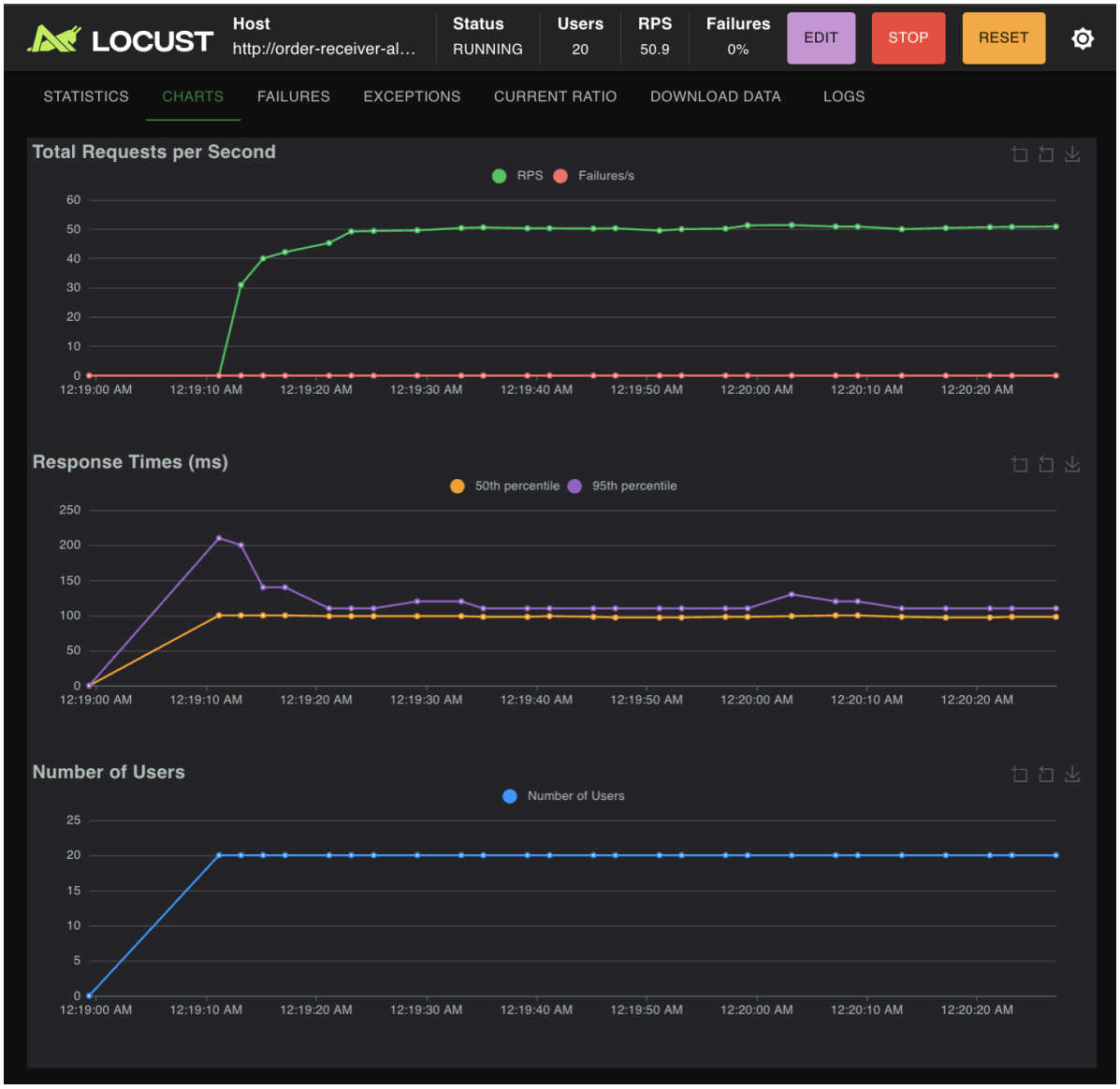
The queue flatlined at the top , it did not drain at all during the test. With 1 worker, it would take over 3 hours to fully process the backlog. This is the hidden cost of async: the customer gets an instant acknowledgment, but their order may not actually be processed for hours. Customer service starts getting calls: *"Where's my order confirmation?"*

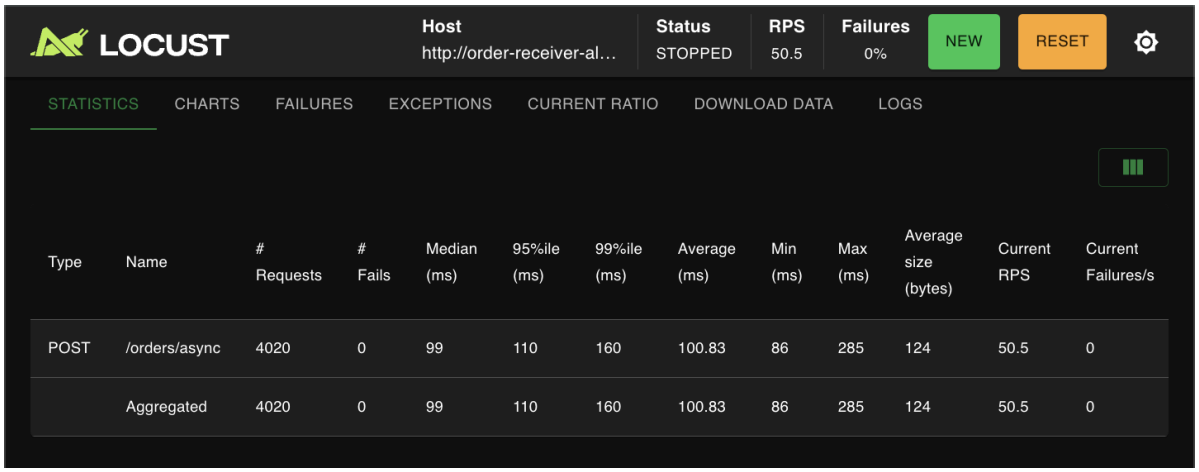
This reveals that accepting orders and processing orders are two separate problems. Fixing one without the other just shifts the bottleneck downstream.

Phase 5

workers:5

Name: Manasha Sekar





Cloud Watch

Name: Manasha Sekar

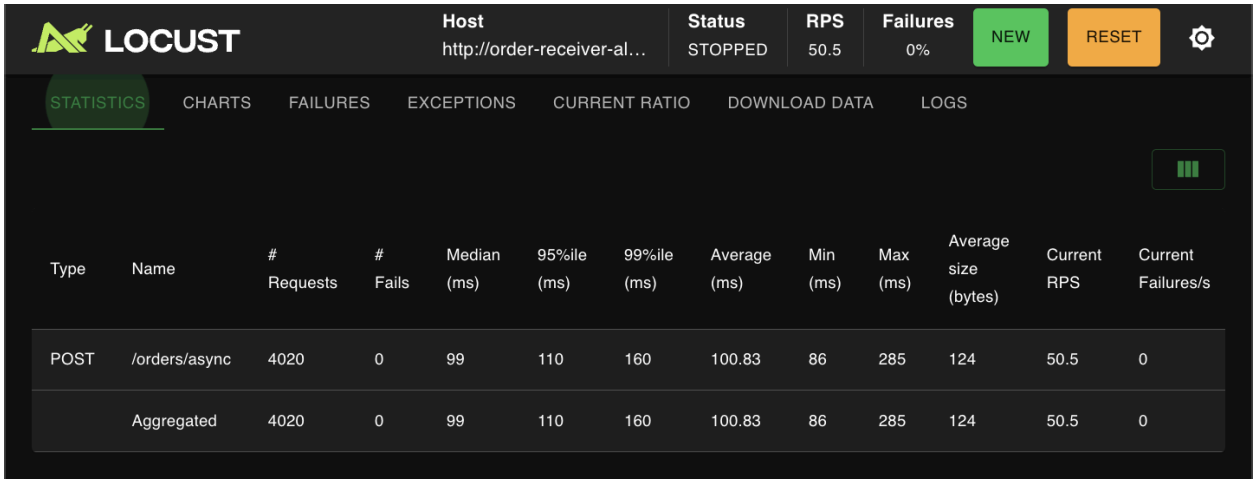


- RPS: 50.5
- Failures: 0%
- Avg Response Time: ~100ms
- Processing rate: 5 ÷ 3 = 1.67 orders/second
- Peak queue depth: ~3,870 messages
- Time to clear backlog: ~38 minutes

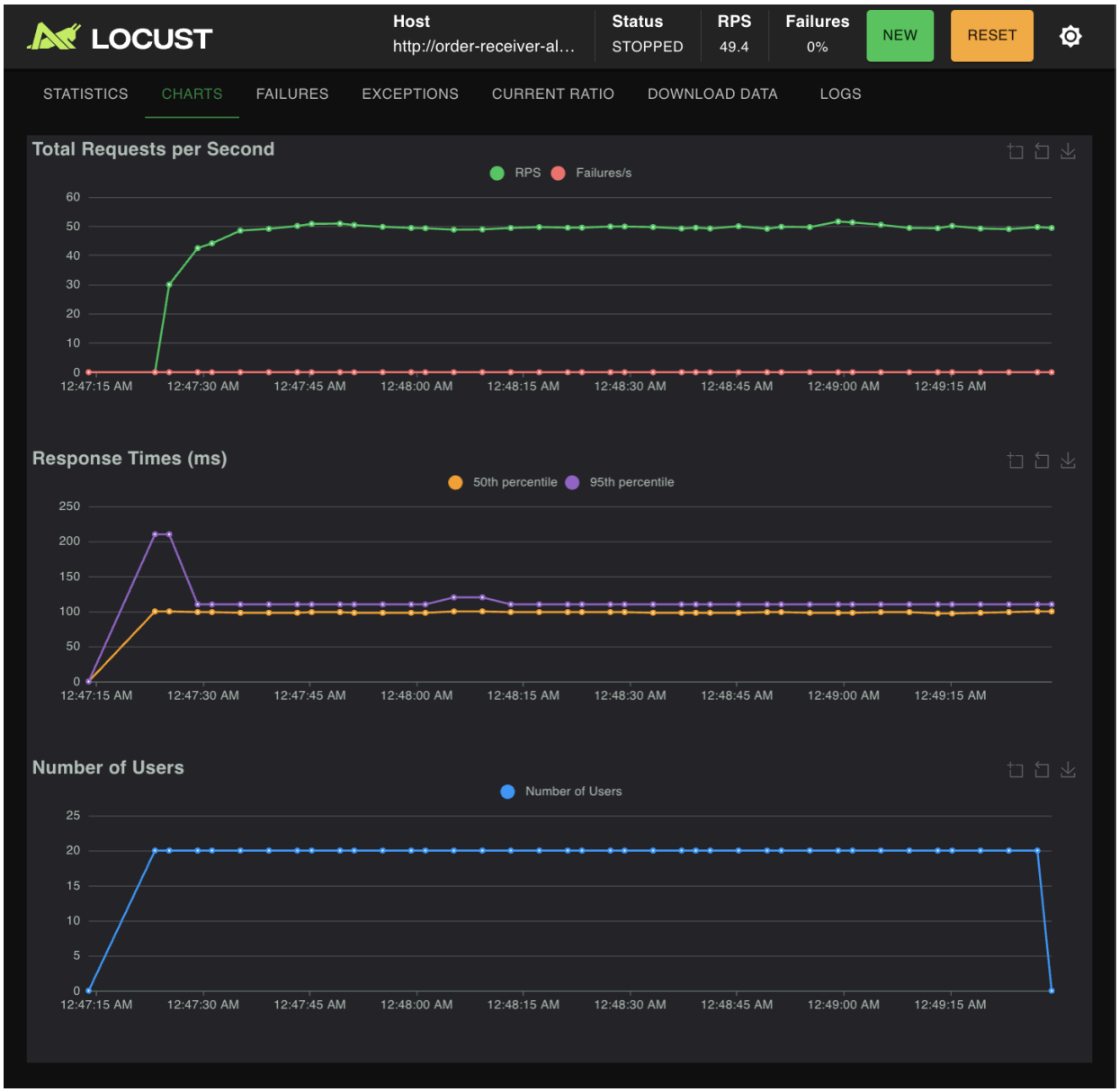
Workers: 20

Ran it for 2 mins.

Name: Manasha Sekar



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	4020	0	99	110	160	100.83	86	285	124	50.5	0
	Aggregated	4020	0	99	110	160	100.83	86	285	124	50.5	0



Cloud Watch:

Name: Manasha Sekar

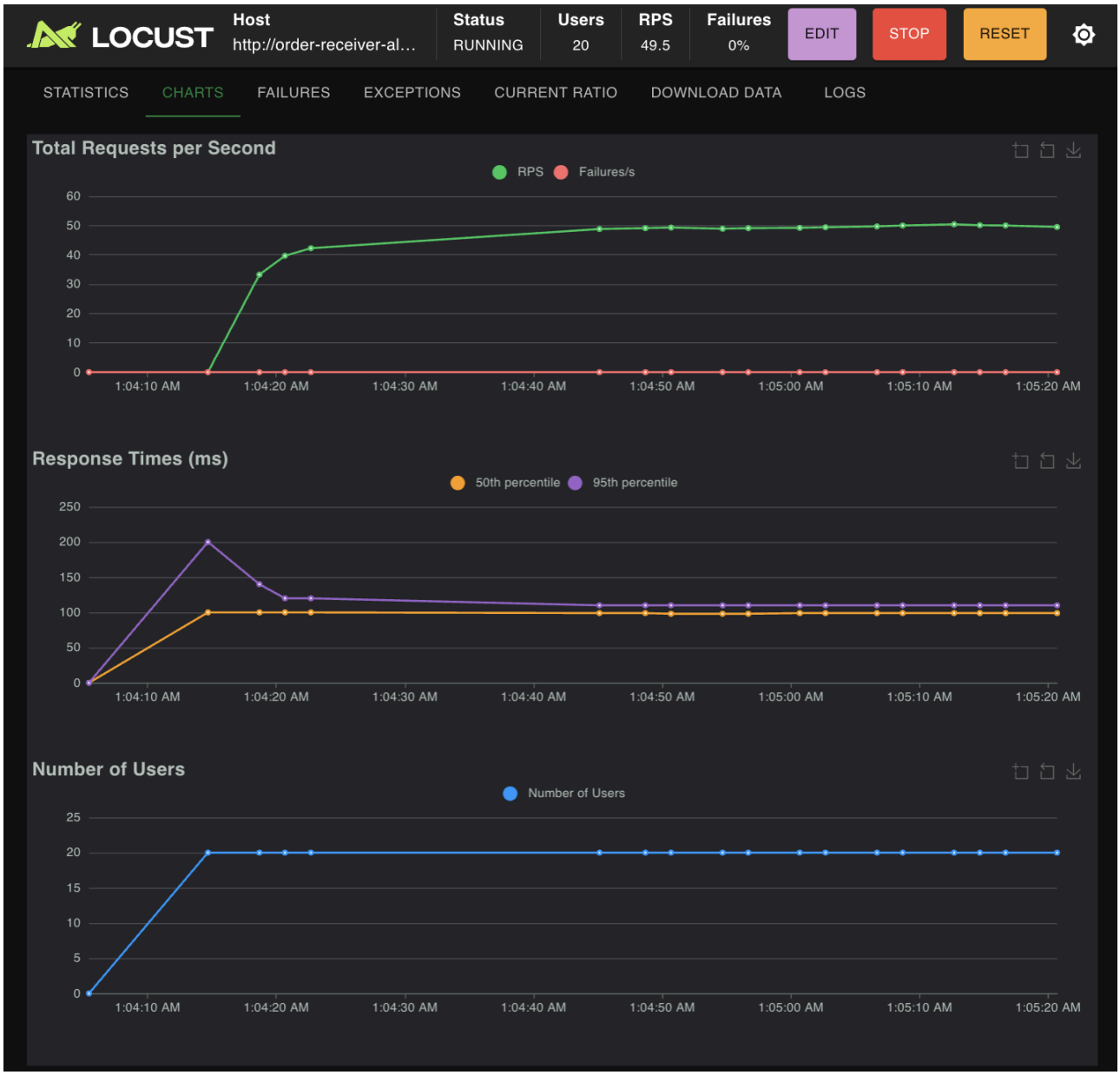


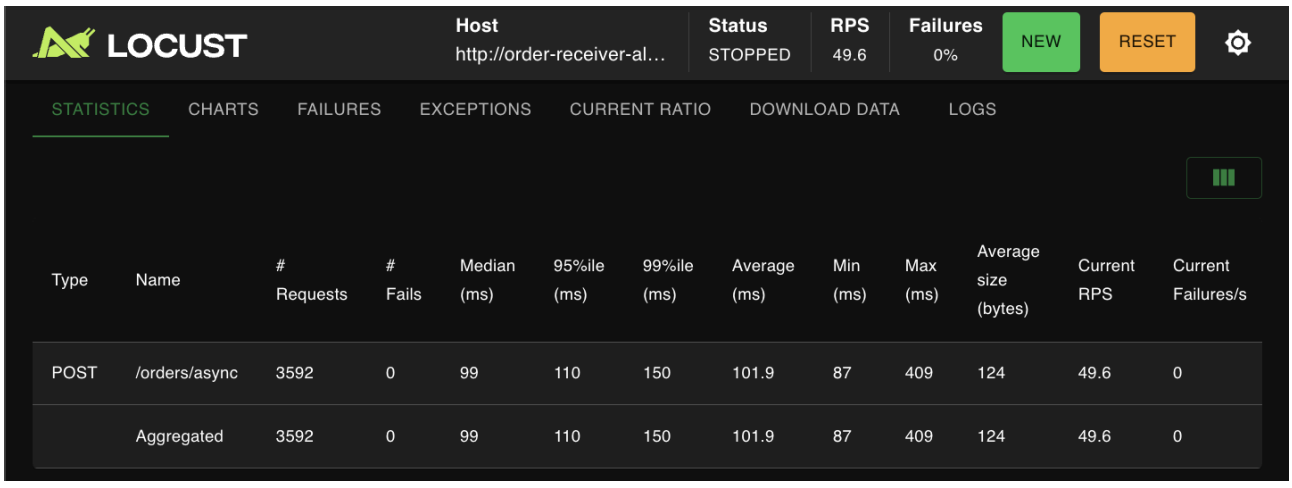
Test Configuration: 20 users, 120 seconds, spawn rate 10/s

- RPS: 50.5
- Failures: 0%
- Avg Response Time: ~100ms
- Processing rate: 20 ÷ 3 = 6.67 orders/second
- Peak queue depth: ~3,870 messages (normalized to 60s)
- Time to clear backlog: ~10 minutes

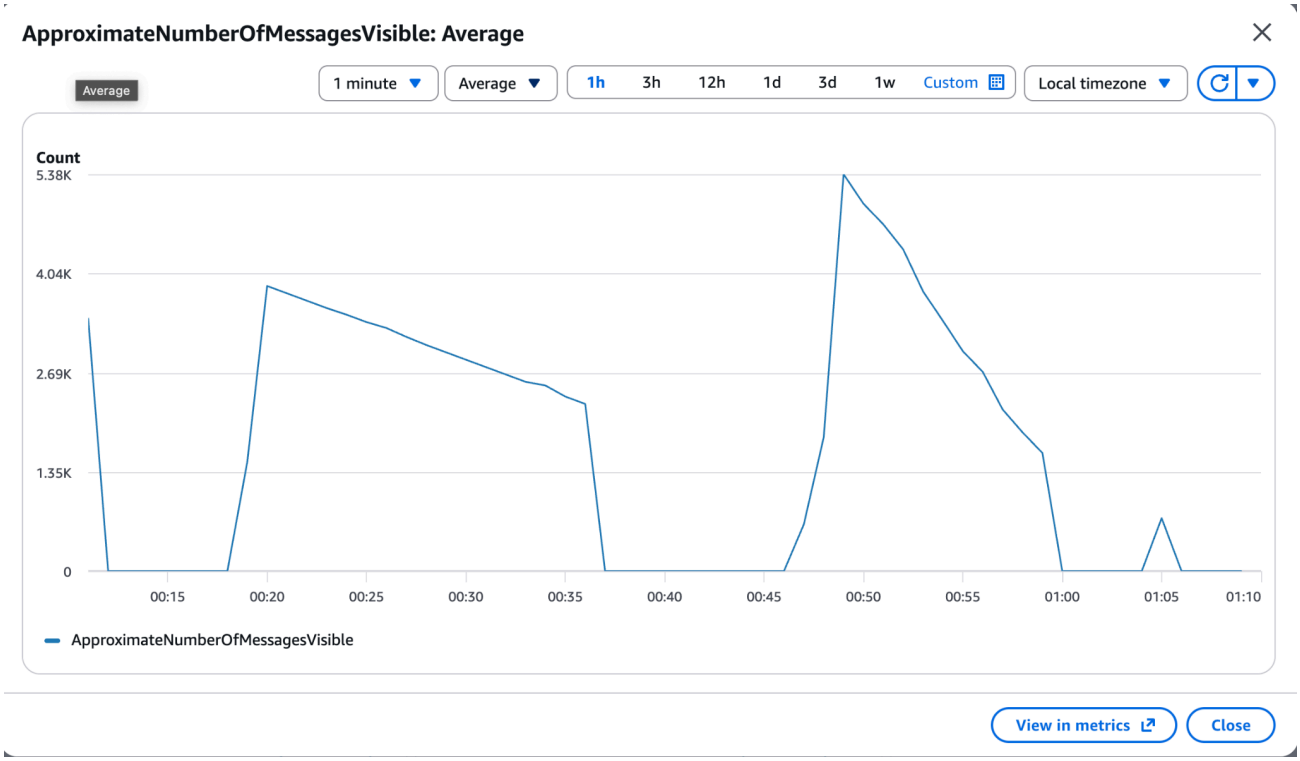
Name: Manasha Sekar

Workers:100





Name: Manasha Sekar



- RPS: 49.6
- Failures: 0%
- Avg Response Time: ~101ms
- Processing rate: 100 ÷ 3 = 33 orders/second
- Peak queue depth: ~3,870 messages (normalized to 60s)
- Time to clear backlog: ~2 minutes

Summary

Workers	Processing Rate	Peak Queue	Time to Clear
1	0.33/s	~3,870	~192 minutes
5	1.67/s	~3,870	~38 minutes
20	6.67/s	~3,870	~10 minutes
100	33/s	~3,870	~2 minutes

To fully prevent queue buildup at 50 orders/second, you would need at least **150 workers** (150 ÷ 3 = 50/s). However, a single ECS task with 256 CPU units and 512MB memory would be heavily throttled well before reaching that number, making horizontal scaling (more ECS tasks) or Lambda a more practical solution.

Name: Manasha Sekar

Analysis Questions

1. How many times more orders did your asynchronous approach accept compared to your synchronous approach?

The async endpoint accepted 3,872 orders whereas 26 with the synchronous approach roughly 149x more orders in the same 60-second window. The sync endpoint was bottlenecked by the 3-second payment verification, limiting it to 0.33 orders/second regardless of how many users were sending requests. The async endpoint decoupled acceptance from processing, allowing it to sustain ~50 orders/second with sub-100ms response times throughout the flash sale.

---

2. What causes queue buildup and how do you prevent it?

Queue buildup occurs when the rate of incoming messages exceeds the rate at which workers can process them. In our case, the async endpoint accepted ~50 orders/second while a single worker could only process 0.33 orders/second — a gap of ~49.67 messages/second. The queue grew to ~3,870 messages in just 60 seconds and would have taken over 3 hours to drain with 1 worker.

The solution is to match processing capacity to the incoming rate. This can be done by scaling the number of worker goroutines (as shown in Phase 5), scaling ECS tasks horizontally, or using auto-scaling based on the CloudWatch metric. At 50 orders/second with a 3-second processing time, at least 150 concurrent workers are needed to prevent buildup entirely.

---

3. When would you choose sync vs async in production?

Choose **synchronous** processing when the result is needed immediately by the caller. Sync is simpler to reason about, easier to debug, and provides immediate feedback on success or failure. It works well when the operation is fast and load is predictable.

Choose **asynchronous** processing when the operation is slow, the result is not needed immediately, or the system must absorb unpredictable bursts of traffic. Order processing, email sending, report generation, and video transcoding are all good candidates.
The trade-off is added complexity: you need a queue, background workers, and a way to notify users when processing completes. You also lose the immediate guarantee that the operation succeeded, a 202 Accepted only means the order was received, not that payment was actually verified.

Name: Manasha Sekar

PART III

```
(base) manashasekar@Manashas-MacBook-Air BSDS % for i in {1..10}; do
  curl -X POST http://order-receiver-alb-272010506.us-west-2.elb.amazonaws.com/orders/async \
    -H "Content-Type: application/json" \
    -d "{\"customer_id\": $i, \"items\": [{\"name\": \"shirt\", \"quantity\": 1, \"price\": 29.99}]}"
  echo ""
done
{"message":"order received, processing in background","order_id":"af87a5f5-160a-4d15-8cfe-f42a0ae478f7","status":"pending"}

{"message":"order received, processing in background","order_id":"a0c2a5d0-1c3b-4624-9db7-242314c20436","status":"pending"}

{"message":"order received, processing in background","order_id":"ed94f021-7393-4b52-9017-12db81abcc5e","status":"pending"}

{"message":"order received, processing in background","order_id":"16f1df82-e4fe-4c67-b9c3-345180b4694a","status":"pending"}

{"message":"order received, processing in background","order_id":"3aacc863-8350-4833-ab84-18bd7a72daa9","status":"pending"}

{"message":"order received, processing in background","order_id":"ad4de481-88f0-4111-ad2a-015b632cda0c","status":"pending"}

{"message":"order received, processing in background","order_id":"0075913a-c00c-4961-addd-140278bdd5b6","status":"pending"}

{"message":"order received, processing in background","order_id":"10f2a46e-2f6a-4ade-a81a-76cc58ba5d65","status":"pending"}

{"message":"order received, processing in background","order_id":"1f305975-4097-4c90-bf88-04a30cac4671","status":"pending"}

{"message":"order received, processing in background","order_id":"10109efa-559d-4f13-94b6-2bd5cbd2d3a5","status":"pending"}
```

```
(base) manashasekar@Manashas-MacBook-Air BSDS % aws logs tail /aws/lambda/order-lambda --follow --region us-west-2
2026-03-16T06:08:38.693000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e START RequestId: 29c3f02f-f880-4a44-ba8a-505112e99f93 Version: $LATEST
2026-03-16T06:08:38.695000+00:00 2026/03/16/[$LATEST]43a20fb07cc24d6f9e35588ad65907e9 START RequestId: eefcdeaa-02f5-4904-8f15-3c393e9b36ac Version: $LATEST
2026-03-16T06:08:38.697000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 START RequestId: c22fe879-27c2-45c0-999b-49cc7c2dc926 Version: $LATEST
2026-03-16T06:08:38.698000+00:00 2026/03/16/[$LATEST]a78f878b78594c499f906693da173f43 START RequestId: b1643665-8e44-4345-a460-b4930c9f98f7 Version: $LATEST
2026-03-16T06:08:38.700000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e 2026/03/16 06:08:38 processing order 3aacc863-8350-4833-ab84-18bd7a72daa9 for customer 5
2026-03-16T06:08:38.700000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 START RequestId: 477d2e5c-89a2-426d-bd58-0072c27adb08 Version: $LATEST
2026-03-16T06:08:38.701000+00:00 2026/03/16/[$LATEST]43a20fb07cc24d6f9e35588ad65907e9 2026/03/16 06:08:38 processing order a0c2a5d0-1c3b-4624-9db7-242314c20436 for customer 2
2026-03-16T06:08:38.703000+00:00 2026/03/16/[$LATEST]a78f878b78594c499f906693da173f43 2026/03/16 06:08:38 processing order 16f1df82-e4fe-4c67-b9c3-345180b4694a for customer 4
2026-03-16T06:08:38.703000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 2026/03/16 06:08:38 processing order 10f2a46e-2f6a-4ade-a81a-76cc58ba5d65 for customer 8
2026-03-16T06:08:38.704000+00:00 2026/03/16/[$LATEST]c8995b67b4414c19830ba6f1444fbb2d START RequestId: ee8d54c0-a499-4be4-b00a-42c5a8511f12 Version: $LATEST
2026-03-16T06:08:38.705000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 2026/03/16 06:08:38 processing order af87a5f5-160a-4d15-8cfe-f42a0ae478f7 for customer 1
2026-03-16T06:08:38.711000+00:00 2026/03/16/[$LATEST]c8995b67b4414c19830ba6f1444fbb2d 2026/03/16 06:08:38 processing order 0075913a-c00c-4961-addd-140278bdd5b6 for customer 7
2026-03-16T06:08:38.810000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad START RequestId: 81ef48d7-2ad7-49c5-bc98-00a2bc39b3dd Version: $LATEST
2026-03-16T06:08:38.815000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad 2026/03/16 06:08:38 processing order ad4de481-88f0-4111-ad2a-015b632cda0c for customer 6
2026-03-16T06:08:38.968000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb START RequestId: 31fa0520-4414-4e91-8387-12e672170f27 Version: $LATEST
2026-03-16T06:08:38.974000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb 2026/03/16 06:08:38 processing order 1f305975-4097-4c90-bf88-04a30cac4671 for customer 9
2026-03-16T06:08:39.155000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d START RequestId: e3a21687-79ab-4956-862d-c6fc2b3ea4b5 Version: $LATEST
2026-03-16T06:08:39.164000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d 2026/03/16 06:08:39 processing order 10109efa-559d-4f13-94b6-2bd5cbd2d3a5 for customer 10
2026-03-16T06:08:41.702000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e 2026/03/16 06:08:41 completed order 3aacc863-8350-4833-ab84-18bd7a72daa9
2026-03-16T06:08:41.703000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 2026/03/16 06:08:41 completed order 10f2a46e-2f6a-4ade-a81a-76cc58ba5d65
2026-03-16T06:08:41.704000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e END RequestId: 29c3f02f-f880-4a44-ba8a-505112e99f93
2026-03-16T06:08:41.704000+00:00 2026/03/16/[$LATEST]4e27cdfaa85c4a25bd40c1a3f01d923e REPORT RequestId: 29c3f02f-f880-4a44-ba8a-505112e99f93        Duration: 3010.20 ms    Billed Durati
on: 3708 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 697.47 ms
2026-03-16T06:08:41.705000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 END RequestId: c22fe879-27c2-45c0-999b-49cc7c2dc926
2026-03-16T06:08:41.705000+00:00 2026/03/16/[$LATEST]304c616572354aedbf77b73950c71ce7 REPORT RequestId: c22fe879-27c2-45c0-999b-49cc7c2dc926        Duration: 3007.13 ms    Billed Durati
on: 3144 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 136.24 ms
2026-03-16T06:08:41.705000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 2026/03/16 06:08:41 completed order af87a5f5-160a-4d15-8cfe-f42a0ae478f7
2026-03-16T06:08:41.707000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 END RequestId: 477d2e5c-89a2-426d-bd58-0072c27adb08
2026-03-16T06:08:41.707000+00:00 2026/03/16/[$LATEST]f81fa4f414d54ff5bdcc0b860e5da740 REPORT RequestId: 477d2e5c-89a2-426d-bd58-0072c27adb08        Duration: 3007.21 ms    Billed Durati
on: 4292 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 1284.70 ms
2026-03-16T06:08:41.816000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad 2026/03/16 06:08:41 completed order ad4de481-88f0-4111-ad2a-015b632cda0c
2026-03-16T06:08:41.818000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad END RequestId: 81ef48d7-2ad7-49c5-bc98-00a2bc39b3dd
2026-03-16T06:08:41.818000+00:00 2026/03/16/[$LATEST]8eaa659544a94eb0b7b0d101dcc2acad REPORT RequestId: 81ef48d7-2ad7-49c5-bc98-00a2bc39b3dd        Duration: 3007.60 ms    Billed Durati
on: 3481 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 472.88 ms
2026-03-16T06:08:41.976000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb 2026/03/16 06:08:41 completed order 1f305975-4097-4c90-bf88-04a30cac4671
2026-03-16T06:08:41.978000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb END RequestId: 31fa0520-4414-4e91-8387-12e672170f27
2026-03-16T06:08:41.978000+00:00 2026/03/16/[$LATEST]145f5a3f1c3c4bd8b812319abeee56bb REPORT RequestId: 31fa0520-4414-4e91-8387-12e672170f27        Duration: 3009.61 ms    Billed Durati
on: 3087 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 76.72 ms
2026-03-16T06:08:42.166000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d 2026/03/16 06:08:42 completed order 10109efa-559d-4f13-94b6-2bd5cbd2d3a5
2026-03-16T06:08:42.168000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d END RequestId: e3a21687-79ab-4956-862d-c6fc2b3ea4b5
2026-03-16T06:08:42.168000+00:00 2026/03/16/[$LATEST]cf0fba8446ca4853b43412ea3c2b584d REPORT RequestId: e3a21687-79ab-4956-862d-c6fc2b3ea4b5        Duration: 3012.70 ms    Billed Durati
on: 3089 ms  Memory Size: 512 MB     Max Memory Used: 20 MB     Init Duration: 76.01 ms
```

```
(base) manashasekar@Manashas-MacBook-Air BSDS % aws logs tail /aws/lambda/order-lambda --follow --region us-west-2
(base) manashasekar@Manashas-MacBook-Air BSDS % curl -X POST http://order-receiver-alb-272010506.us-west-2.elb.amazonaws.com/orders/async \
  -H "Content-Type: application/json" \
  -d '{"customer_id": 123, "items": [{"name": "shirt", "quantity": 1, "price": 29.99}]}'
{"message":"order received, processing in background","order_id":"5c4fa7c2-3d9d-43d0-80e4-49eb285bcf0d","status":"pending"}
```

Name: Manasha Sekar



1. Deployed Lambda Function

A Lambda function was deployed using the provided.al2 runtime with 512MB memory, subscribed directly to the existing SNS topic order-processing-events. Orders flow from the API -> SNS -> Lambda with no SQS queue in between. The same 3-second payment simulation runs inside the Lambda handler.

2. Cold Start Observations

10 orders were sent simultaneously to trigger concurrent Lambda invocations. Since all 10 fired at the same time, Lambda spun up 10 fresh instances in parallel, every invocation was a cold start.

Cold start results (10 concurrent invocations):

Request	Duration	Init Duration
customer 5	3010ms	697ms
customer 8	3007ms	136ms

customer 1	3007ms	1284ms
customer 2	3009ms	1225ms
customer 3	3010ms	1083ms
customer 4	3009ms	747ms
customer 6	3007ms	472ms
customer 7	3008ms	325ms
customer 9	3009ms	76ms
customer 10	3012ms	76ms

- Cold start overhead ranged from **76ms to 1284ms**
- High variance because all 10 instances initialized simultaneously competing for resources
- Memory used was only **20MB out of 512MB**.

Warm start (after ~8 minutes idle):

REPORT Duration: 3003.66ms, Max Used: 20MB

No Init Duration, Lambda reused an existing instance. The warm start added **0ms overhead** on top of the 3 second processing time.

Cold starts occur: on first invocation, after ~5+ minutes idle, and when concurrent requests exceed the number of warm instances.

3. Cost Calculation

Current ECS cost:

- 2 ECS tasks × $8.50/month = **$17/month** (always running)

Name: Manasha Sekar

Lambda cost (AWS free tier):

- Free tier: 1M requests + 400K GB-seconds/month
- Per order: 3 seconds × 0.5GB = 1.5 GB-seconds
- Free orders per month: 400,000 ÷ 1.5 = **~267,000 orders FREE**

Monthly Volume	Lambda Cost	ECS Cost
10,000 orders	$0 (free tier)	$17
100,000 orders	$0 (free tier)	$17
267,000 orders	$0 (free tier)	$17
1,000,000 orders	~$5.50	$17

Break-even: Lambda costs more than ECS only beyond ~1.7M orders/month.

4. Switch Recommendation

For our startup at current order volumes, switching to Lambda makes sense. Lambda eliminated all operational overhead, no SQS queue to monitor, no worker goroutine tuning, no ECS health checks. The cost advantage is compelling: at under 267,000 orders per month Lambda is completely free. Cold starts ranged from 76ms to 1284ms but averaged around 400ms on top of a 3 second payment processing time. The main trade-off is losing SQS guarantees: SNS retries Lambda only twice before discarding the message, whereas SQS provides configurable retries, dead-letter queues, and visibility timeouts. For a startup under 267K orders/month where operational simplicity matters more than message durability guarantees, Lambda is the right choice. As the business scales past 1M+ orders/month or requires stricter delivery guarantees, revisiting the SQS + ECS architecture would be warranted.